

A key difference between a traditional Web application(monolithic) and the RESTful web service is the way that the HTTP response body is created.

Instead of using a view technology(JSP / Thymeleaf) to perform server-side rendering of the data to HTML, typically a RESTful web service controller simply populates and returns a java object(data). The object data will be written directly to the HTTP response as JSON/XML/Text

To do this, the `@ResponseBody` annotation on the ret type of the request handling method tells Spring MVC that it does not need to render the java object through a server-side view layer.

Instead it tells that the java object returned is the response body, and should be written out directly.

The java object must be converted to JSON. Thanks to Spring's HTTP message converter support, you don't need to do this conversion manually. Because Jackson Jar is on the classpath, SC can automatically convert the java object to JSON & vice versa (using 2 annotations `@ResponseBody` & `@RequestBody`)

API --Starting point

`o.s.http.converter.HttpMessageConverter<T>`

--Interface that specifies a converter that can convert from and to HTTP requests and responses.

T --type of request/response body.

Implementation classes

1. `o.s.http.converter.xml.Jaxb2RootElementHttpMessageConverter`

-- Implementation of `HttpMessageConverter` that can read and write XML using JAXB2.(Java architecture for XML binding)

2. `o.s.http.converter.json.`

`MappingJackson2HttpMessageConverter`

--Implementation of `HttpMessageConverter` that can read and write JSON using Jackson 2.x's `ObjectMapper` class API

Important Annotations

1. `@ResponseBody` : added implicitly on ret types of all req handling methods.

Applied at : return value of the request handling method , annotated with `@RequestMapping` or `@GetMapping` / `@PostMapping` / `@PutMapping` / `@DeleteMapping`

It is used to marshall(serialize=Java Object --> JSON|XML)) the return value , into the HTTP response body. Spring comes with converters that convert the Java object into a format understandable for a client(text/xml/json)

Vendor for marshalling n later un marshalling (used by default with Spring Boot) : Jackson

eg :

`@Controller`

`@RequestMapping("/employees")`

`public class EmpController`

```
{
    @GetMapping(...)
    public @ResponseBody Emp fetchEmpDetails(int empId)
    {
        //get emp dtls from DB through layers
        return e;
    }
}
```

```
}
```

2. @RestController

Class level annotation

Good news is @RestController = @Controller(at the class level) + @ResponseBody implicitly added on ret types of ALL request handling methods(annotated with @RequestMapping |@GetMapping|@PostMapping|@PutMapping...)

```
eg :
@RestController
@RequestMapping("/employees")
public class EmpController
{
    @GetMapping(...)
    public Emp fetchEmpDetails(int empId)
    {
        //get emp dtls from DB through layers
        return e;
    }.....
}
```

3. @PathVariable --- handles URI templates.(URI variables or path variables)

Where to apply : on the method argument of the request handling method
Purpose : to bind a URI variable(path var) to the the method argument.

eg : URL -- http://host:port/products/1234 , method=GET
http://host:port/products/categories/cat_id/10/product_id/1234 , method=GET

Simple eg :
http://host:port/products/1234 , method=GET
In ProductController :
@RestController
@RequestMapping("/products") {
@GetMapping("/{pid}")
public Product getDetails(@PathVariable Long pid)
{fetch product details by its id}

OR for sub resources :

Method of ProductController
@RestController
@RequestMapping("/products/categories/cat_id/{catId}") {
@GetMapping("/productId/{pid}")
public Product getDetails(@PathVariable int catId,@PathVariable(name="pid") int pid1234)
{...}
}

In the above URL , the path variable {pid} is mapped to an int . Therefore all of the URIs such as /products/1 or /products/10 will map to the same method in the controller.

4. The @RequestBody annotation, unmarshalls the HTTP request body into a Java object injected in the

method.

Applied on the method argument of the request handling methods , containing request body

eg : Typically in POST , PUT , PATCH

@RequestBody must be still added on a method argument of request handling method , for unmarshaling(de serialization)

5. @CrossOrigin

Class/Method level annotation , in RestController

What is CORS ?

Cross-Origin Resource Sharing (CORS)

CORS is an HTTP-header based mechanism that allows a server to indicate any origins (domain, scheme, or port) other than its own from which a browser should permit loading of resource(response)

A cross-origin HTTP request is a request to a specific resource, which is located at a different origin, namely a domain, protocol and port, than the one of the client performing the request.

For security reasons, browsers can request several cross-origin resources, including images, CSS, JavaScript files etc. By default, a browser' security model will deny any cross-origin HTTP request performed by client-side scripts.

While this behavior is desired, to prevent different types of Ajax-based attacks, sometimes we need to instruct the browser to allow cross-origin HTTP requests from JavaScript clients with CORS.

eg : React client running on http://localhost:3000, and a Spring Boot RESTful web service API listening at http://host:port/products

In such a case, the client should be able to consume the REST API, which by default would be forbidden.

To make this work , enable CORS by simply annotating the class / methods of the RESTful web service API responsible for handling client requests with the @CrossOrigin annotation

eg : @CrossOrigin(origins = "http://localhost:3000")

@RestController

public class ProductController{....}